Отчет по индивидуальному заданию №3

ФИО: Куделькина Дарья Дмитриевна
Группа: БПИ-253
Вариант: 44

Условие задачи

Предприятие выпускает изделия, состоящие из нескольких деталей. Для каждой детали задан технологический маршрут: последовательность операций на станках определённых типов. Готовые детали поступают в промежуточные накопители, после чего сборочный пост может начать работу только при наличии полного комплекта компонентов. После обработки деталь проходит контроль качества. Годная деталь передаётся дальше, бракованная направляется на повторную обработку или списывается. Вместимость накопителей ограничена, поэтому заполненный выходной буфер способен приостановить станок. Транспортные роботы перемещают детали между стадиями. Требуется разработать консольное приложение, моделирующее сборочную линию.

Входные параметры: перечень изделий и состав комплектов; технологические маршруты деталей; количество станков и сборочных постов; число транспортных роботов; длительность операций; вместимость накопителей; вероятность брака и правила повторной обработки; план выпуска изделий.

Отображаемые события: создание детали, постановка в очередь, начало и окончание операции, контроль качества, обнаружение брака, повторная обработка, перемещение, поступление в комплект, начало сборки и выпуск изделия.

Инварианты: станок обрабатывает не более одной детали; деталь находится на одной стадии или у одного транспортного робота; операция выполняется после всех предыдущих операций маршрута; забракованная деталь не входит в готовое изделие; сборка начинается только при наличии полного допустимого комплекта.

Завершение: после выполнения плана, исчерпания исходных деталей либо признания дальнейшего производства невозможным. Станки, контролёры, сборочные посты, транспортные роботы и детали являются самостоятельными участниками модели.

Состав решения

Выполнены требования всех уровней, поэтому решение состоит из четырёх программ. Все они моделируют одну и ту же линию, выводят одинаковые события и итоги и отличаются только способом взаимодействия процессов.

Программа line_unnamed: единый родительский процесс порождает процессы-участники через fork. Состояние линии хранится в разделяемой памяти POSIX, процессы синхронизируются неименованными семафорами POSIX, которые лежат в этой же разделяемой памяти.

Программа line_named: родительский процесс также порождает участников через fork, состояние линии - в разделяемой памяти, но семафоры именованные.

Программы shm_manager и shm_worker: каждый участник - отдельная программа shm_worker, которая запускается независимо от других, например в своем терминале. Процесс-менеджер shm_manager создаёт разделяемую память и именованные семафоры, участники подключаются к ним по именам.

Программы mq_manager и mq_worker: участники mq_worker также запускаются независимо. Синхронизация - именованные семафоры, а обмен данными идёт через очереди сообщений POSIX. Разделяемой памяти в этой программе нет: состояние линии есть только у менеджера, участники получают и меняют его через сообщения.

Анализ сущностей

Станок, контролёр ОТК, транспортный робот и сборочный пост реализованы как процессы. Каждый из них работает по одному циклу: дождаться работы, взять деталь, некоторое время её обрабатывать и передать дальше. Такой цикл естественно ложится на процесс: ожидание - блокировка на семафоре, обработка - сон процесса на время операции. На каждый экземпляр запускается свой процесс: два фрезерных станка - два процесса.

Склад заготовок через равные промежутки времени выпускает новые заготовки, а когда выпускать нечего, ждёт, не спишут ли деталь, которую нужно заменить. Поэтому склад тоже реализован как отдельный процесс.

Деталь ничего не делает сама, её перемещают и обрабатывают другие. Если сделать каждую деталь процессом, такой процесс всё время будет ждать, пока его возьмут. Число деталей к тому же заранее неизвестно и может достигать сотен. Поэтому деталь реализована как данные - запись с номером, типом, позицией в маршруте и местом нахождения.

Накопители (очереди к станкам, выходные накопители, вход и выход ОТК, комплекты) поведения не имеют. Это общие ресурсы с ограниченной вместимостью, через которые процессы передают друг другу детали. Они реализованы как данные в общем состоянии линии, а свободные места и готовые детали в каждом накопителе считают семафоры.

Изделие - результат работы сборочного поста, поведения не имеет и хранится как счётчик выпуска.

Помимо участников модели есть процесс-менеджер. Он создает объекты IPC, дожидается подключения всех участников, работает «сторожем» - следит за лимитом времени и замечает, что линия встала, - принимает сигналы пользователя, выводит итоги и удаляет объекты IPC.

Итого при базовой конфигурации работают 10 процессов-участников (склад, 5 станков, 1 контролёр, 2 робота, 1 пост) и процесс-менеджер.

Основные сущности программы

Сборочная линия (Line)

Всё состояние линии хранится в одной структуре Line: копия параметров, детали, накопители, станки, контролёры, роботы, посты, счётчики выпуска, статистика, флаги остановки и причина завершения. В первых трёх программах эта структура лежит в разделяемой памяти и видна всем процессам, в четвёртой - только у менеджера.

Внутри Line нет указателей: разделяемая память у разных процессов может оказаться по разным адресам, поэтому накопители, детали и участники обозначаются номерами. Параметры копируются в Line целиком, поэтому независимым участникам не нужен файл конфигурации - всё нужное они узнают у менеджера.

Время в модели измеряется в условных единицах: одна единица модельного времени равна заданному числу миллисекунд реального времени (по умолчанию 50, ключ -d). Операция длительностью 4 единицы означает, что процесс станка спит 4 * 50 = 200 мс. Момент начала моделирования хранится в общем состоянии, а время берется по монотонным часам системы, которые у всех процессов общие.

Деталь (Part)

Деталь - данные. У детали есть тип, номер следующей операции маршрута, число переделок, отметка о прохождении контроля и место, где она находится. Деталь в каждый момент находится ровно в одном месте: на складе, в очереди к станку, на станке, в выходном накопителе станка, у робота, на входе ОТК, у контролёра, на выходе ОТК, в комплекте, на сборке, в готовом изделии или в списании.

Накопитель (Store)

Накопитель - ограниченный буфер деталей. У каждого накопителя есть два семафора. Семафор свободных мест: тот, кто хочет положить деталь, сначала уменьшает его, а тот, кто забрал деталь, увеличивает. Семафор готовых деталей: его увеличивает робот после доставки, а ждёт на нём станок или контролёр.

Операции участников

Любое изменение линии оформлено как операция: «склад выпускает заготовку», «станок берет деталь из очереди», «станок закончил операцию», «контролёр принял решение», «робот взял деталь», «робот доставил деталь», «пост забрал комплект», «пост выпустил изделие», а также «участник подключился» и «участник отключился». Все операции выполняет одна функция line_op, и каждая выполняется целиком, без перерыва. Внутри операции никто не ждёт: семафоры там только увеличиваются или уменьшаются без ожидания (sem_trywait). Ждут сами процессы - между операциями.

Благодаря этому поведение участников описано один раз (файл worker.c) и одинаково во всех четырех программах. Различается только то, как процесс вызывает операцию: сам под замком в разделяемой памяти или запросом менеджеру через

очередь сообщений.

Склад заготовок (процесс warehouse)

Раз в release_interval единиц склад выполняет операцию выпуска: выбирает, деталь какого типа нужна (по очереди между типами, только если заготовки этого типа ещё есть и деталей на линии вместе с уже собранными меньше, чем требует план), создаёт её и увеличивает семафор «у роботов есть работа». Если выпускать нечего, склад засыпает на семафоре «нужна замена», который увеличивает контролер при списании детали.

Станок (процесс machine)

Станок ждёт на семафоре готовых деталей очереди своего типа и берёт первую деталь, проверяя, что следующая операция её маршрута выполняется именно на станке этого типа. Затем он спит на время операции. После операции номер шага маршрута увеличивается, и станок пытается без ожидания занять место в своем выходном накопителе. Если накопитель полон, станок сообщает о приостановке и ждет на семафоре свободных мест, пока робот не заберёт деталь. Время простоя учитывается в статистике.

Контролер ОТК (процесс inspector)

Контролер ждёт деталь на входе ОТК, проверяет её и случайным образом решает, есть ли брак, с вероятностью, заданной для типа детали. Для случайных чисел используется функция rand_r с собственным состоянием у каждого контролера. Годная деталь получает отметку и попадает на выход ОТК. Бракованная либо возвращается на заданную операцию маршрута, либо списывается, и тогда контролер будит склад, чтобы тот выпустил замену. Если выход ОТК полон, контролер ждёт места так же, как станок.

Транспортный робот (процесс robot)

Робот спит на общем семафоре «у роботов могла появиться работа», который увеличивает любая операция, после которой роботу может найтись дело. Проснувшись, робот ищет деталь, которую можно везти: сначала на выходе ОТК, затем в выходных накопителях станков, затем на складе. Деталь везут только туда, где для неё есть место: место назначения бронируется заранее уменьшением семафора свободных мест без ожидания. Затем робот спит время перевозки и кладёт деталь в забронированное место, после чего будит потребителя: станок или ОТК - через семафор готовых деталей, сборочные посты - через семафор «комплекты изменились».

Сборочный пост (процесс station)

Пост спит на семафоре «комплекты изменились». Проснувшись, он проверяет, есть ли полный комплект для изделия, план по которому еще не выполнен, и забирает весь комплект одной операцией. Поэтому два поста не могут разобрать один комплект пополам. После сборки пост выпускает изделие, и если план выполнен по всем

изделиям, останавливает линию.

Сценарий взаимодействия процессов

Общая схема движения детали:

[склад] -(робот)-> очередь станка A -> [станок A] -> выход станка A -(робот)-> ... ->
вход ОТК -> [контролер] -> выход ОТК:
годна -(робот)-> комплект у сборочного поста
переделка -(робот)-> очередь станка операции k
брак без переделки -> списание
полный комплект -> [сборочный пост] -> изделие

В квадратных скобках - процессы, остальное - накопители в общем состоянии линии.

Семафоры

В программе три вида семафоров. Замок S_LOCK - двоичный семафор с начальным значением 1: в программах на разделяемой памяти процесс захватывает его на время операции, поэтому общее состояние никогда не меняют два процесса сразу. Семафоры ресурсов - по два у каждого накопителя (свободные места и готовые детали): это классическая схема «производитель - потребитель» с ограниченным буфером. Семафоры событий - «у роботов есть работа», «комплекты изменились», «нужна замена» и «старт»: проснувшийся процесс выполняет операцию, сам проверяет, есть ли для него работа, и если нет, снова засыпает. Лишнее пробуждение безопасно, а пропустить нужное невозможно, потому что увеличение семафора запоминается, даже если в этот момент никто не ждёт.

Все семафоры обозначаются номерами: для базовой конфигурации их 35. Номера, на которых нужно ждать конкретному участнику, он получает от менеджера при подключении.

Два правила соблюдаются во всех программах: процесс ждёт на семафоре только вне операции (иначе, держа замок, он остановил бы всю линию), и долгая работа (операция, перевозка, сборка) тоже выполняется вне операции. Поэтому процессы работают параллельно и почти не мешают друг другу.

Пример передачи детали от станка LATHE-1 к станку GRIND-1:

1. Станок GRIND-1 ждёт работу на семафоре готовых деталей очереди GRIND.

2. Станок LATHE-1 спит время операции, затем выполняет операцию «закончил»: занимает место в своем выходном накопителе (sem_trywait), кладёт деталь и увеличивает семафор «у роботов есть работа».

3. Робот просыпается, выполняет операцию «взять»: находит деталь, бронирует место в очереди GRIND (sem_trywait), забирает деталь и увеличивает семафор свободных мест выхода LATHE-1.

4. Робот спит время перевозки, затем выполняет операцию «доставить»: кладёт

деталь в очередь GRIND и увеличивает её семафор готовых деталей. 5. Станок GRIND-1 просыпается и операцией «взять» забирает деталь.

Подключение и старт

Каждый участник при запуске выполняет операцию «подключиться» с указанием своей роли и номера. Менеджер (в первых двух программах - родитель) проверяет, что такой участник нужен и ещё не подключён, сообщает ему номера семафоров и длительность его работы. После этого участник ждёт на семафоре «старт». Когда подключились все, менеджер запускает часы модели и увеличивает семафор «старт» столько раз, сколько участников. Поэтому независимые процессы можно запускать в любом порядке и с любыми паузами - моделирование начнётся только тогда, когда соберётся вся линия. Если участник запущен раньше менеджера, он ждёт, пока менеджер создаст объекты IPC.

Остановка

Когда нужно завершить работу (план выполнен, получен сигнал, сторож обнаружил, что линия встала), ставится флаг остановки, записывается причина и увеличиваются все семафоры, на которых может спать какой-нибудь процесс. Каждый процесс, проснувшись, выполняет очередную операцию, получает ответ «пора завершаться», выполняет операцию «отключиться» и выходит. Менеджер дожидается всех участников (родитель - вызовом waitpid, менеджер независимых процессов - по счётчику подключенных участников), проверяет правила, выводит итоги и удаляет объекты IPC.

Сигналы

Сигналы принимает менеджер. Обработчик, установленный функцией sigaction, только ставит флаг, а действует главный цикл менеджера: SIGINT и SIGTERM останавливают линию, SIGUSR1 печатает текущее состояние линии, SIGUSR2 меняет условие завершения - склад перестает выпускать новые заготовки, а детали, которые уже на линии, доделываются. Участники игнорируют SIGINT, поэтому Ctrl+C, который терминал посылает всей группе процессов, обрабатывает только менеджер, и линия останавливается согласованно. SIGTERM, посланный одному участнику, завершает только его: он корректно отключается, а менеджер, заметив, что линия встала без него, завершает работу с соответствующей причиной.

Программа 1: неименованные семафоры в разделяемой памяти

Родительский процесс line_unnamed создаёт разделяемую память POSIX (shm_open, ftruncate, mmap) размером со структуру линии и массив семафоров. Семафоры создаются функцией sem_init прямо в разделяемой памяти со вторым аргументом pshared = 1, который разрешает пользоваться ими из разных процессов.
Затем родитель порождает fork-ом всех участников. Дочерние процессы наследуют отображение памяти, поэтому сразу видят и линию, и семафоры. Операцию процесс выполняет сам, захватив замок S_LOCK.

При завершении любым способом родитель дожидается детей, уничтожает семафоры (sem_destroy) и удаляет разделяемую память (munmap, shm_unlink).

Программа 2: именованные семафоры

Программа line_named устроена так же, но семафоры именованные: семафор номер i называется /asm_line_s<i> и создаётся функцией sem_open с флагом O_CREAT. Процессы открывают семафоры по имени при первом обращении. При завершении семафоры закрываются и удаляются функцией sem_unlink. Обе программы собираются из одного исходного кода: различается только файл с реализацией семафоров (sync_unnamed.c или sync_named.c) с одинаковым интерфейсом.

Программа 3: независимые процессы, разделяемая память

Программа shm_manager создает разделяемую память и именованные семафоры, а участники - отдельные программы shm_worker, которые запускаются командой вида ./bin/shm_worker machine 0 и ничего не знают о менеджере, кроме имени объектов IPC (ключ -n). Участник открывает разделяемую память по имени (shm_open без O_CREAT, при отсутствии ждёт её появления), отображает её mmap, открывает именованные семафоры по именам и сам выполняет операции под замком. События он выводит сам, а журнал открывает с флагом O_APPEND, поэтому строки разных процессов не перемешиваются.

Список нужных участников менеджер выводит по ключу -L. Скрипт tests/run_independent.sh запускает менеджера и всех участников отдельными командами, избавляя от ручного запуска десятка программ.

Программа 4: независимые процессы, очереди сообщений

В программе mq_manager состояние линии хранится только в памяти менеджера, разделяемой памяти нет. Менеджер создает очередь запросов /asm_line_req, а каждый участник mq_worker - свою очередь ответов /asm_line_r<номер>. Чтобы выполнить операцию, участник отправляет в очередь запросов сообщение (операция, номер участника, куда отвечать) и ждёт ответ - результат операции. Менеджер принимает запросы функцией mq_timedreceive и выполняет их строго по одному, поэтому замок здесь не нужен: его роль играет очередь запросов.

Ожидание по-прежнему организовано на именованных семафорах: участник ждёт на семафоре, а менеджер, выполняя операцию, увеличивает семафоры тех, кого нужно разбудить. Так, станок ждёт деталь на семафоре готовых деталей своей очереди, а менеджер увеличивает этот семафор, когда выполняет запрос робота «доставить». Таймаут ожидания запроса равен одной единице времени, и между запросами менеджер проверяет сигналы и работает сторожем.

При завершении менеджер удаляет очередь запросов и семафоры, участники - свои очереди ответов. Перед запуском менеджер удаляет очереди, оставшиеся от

аварийно прерванного запуска, а участник не запустится, если очередь с его номером уже есть, то есть такой участник уже работает.

Правила, которые соблюдаются всегда

Станок обрабатывает не более одной детали, деталь находится на одной стадии или у одного робота, операция выполняется только после всех предыдущих операций маршрута, забракованная деталь не входит в готовое изделие, сборка начинается только при наличии полного комплекта.

Эти правила обеспечиваются тем, что любое перемещение детали - это одна операция, выполняемая целиком, а вместимость накопителей не может быть превышена, потому что положить деталь можно, только получив место через семафор свободных мест. Кроме того, правила проверяются. Во время работы станок проверяет, что пришедшая к нему деталь должна выполнять операцию именно на станке этого типа и что он свободен; пост проверяет, что каждая деталь комплекта прошла контроль. После завершения всех процессов выполняется полная проверка: для каждой детали считается, в скольких местах она найдена. Деталь на линии должна встретиться ровно один раз, вошедшая в изделие или списанная - ни разу. Также проверяется, что ни один накопитель не переполнен. О любом нарушении программа сообщает событием «!ИНВАРИАНТ».

Входные данные

Без указания файла используется встроенная конфигурация - участок выпуска редукторов (REDUCER), который собирается из вала (SHAFT), двух шестерён (GEAR) и корпуса (HOUSING) на токарном (LATHE), двух фрезерных (MILL), сверлильном (DRILL) и шлифовальном (GRIND) станках. Файл конфигурации читается системными вызовами open, read и close, символ # начинает комментарий:

```
machine LATHE 1
part SHAFT raw 6 defect 0.10 rework 2 max_rework 1 route LATHE:4 GRIND:3
product REDUCER plan 4 kit SHAFT:1 GEAR:2 HOUSING:1
robots 2
```

Строка machine задаёт тип станка и количество станков данного типа - от него зависит число процессов-станков. Строка part описывает тип детали: запас заготовок (raw), вероятность брака (defect), с какой операции переделывать брак (rework, или none - сразу списывать), сколько раз можно переделывать (max_rework) и маршрут (route) в виде «станок:длительность». Строка product задаёт изделие: план выпуска (plan) и состав комплекта (kit). Остальные параметры: число роботов и время перевозки (robots, transport_time), число контролёров и время проверки (inspectors, inspect_time), число постов и время сборки (stations, assembly_time), вместимости накопителей, интервал выпуска заготовок (release_interval), начальное значение генератора случайных чисел (seed) и лимит модельного времени (max_time). Число процессов каждого вида задаётся этими же параметрами.

При синтаксической ошибке программа сообщает имя файла, номер строки и

причину. Затем проверяется смысл данных, и если изделие собрать заведомо невозможно, программа сообщает об этом, не создавая процессов и объектов IPC.

Интерфейс

Запуск

cd HW-3
make
./bin/line_unnamed [-c файл] [-s seed] [-d мс] [-l журнал] [-q] [-t лимит] [-n имя]
./bin/line_named [те же ключи]
./bin/shm_manager [те же ключи] [-L] ./bin/shm_worker [-n имя] <роль> <номер>
./bin/mq_manager [те же ключи] [-L] ./bin/mq_worker [-n имя] <роль> <номер>
./tests/run_independent.sh shm|mq [ключи менеджера]

Ключ -c задаёт файл конфигурации. Ключ -s задаёт начальное значение генератора случайных чисел. Ключ -d задаёт, сколько миллисекунд длится единица модельного времени: чем больше значение, тем медленнее идёт моделирование и тем удобнее следить за событиями. Ключ -l задаёт файл журнала (по умолчанию line.log, «-l -» - без журнала). Ключ -q включает тихий режим, когда на экран выводятся только итоги. Ключ -t ограничивает модельное время. Ключ -n задаёт имя объектов IPC, чтобы на одной машине можно было одновременно запустить несколько линий. Ключ -L выводит список нужных участников («machine 0», «robot 1» и т. д.). Роли участников: warehouse, machine, inspector, robot, station. Скрипт tests/demo.sh запускает любую из программ медленно (150 мс на единицу).

Управление во время работы - сигналами менеджеру из другого терминала:

kill -USR1 <pid> показать состояние линии
kill -USR2 <pid> мягкая остановка
kill <pid> остановка (SIGTERM), то же, что Ctrl+C

Что выводит программа

В начале выводится сводка параметров, затем подключение процессов участников и запуск линии:

[t= 0] СИСТЕМА Склад процесс подключился (1 из 10)
[t= 0] СИСТЕМА LATHE-1 процесс подключился (2 из 10)
...
[t= 0] СИСТЕМА ПОСТ-1 процесс подключился (10 из 10)
[t= 0] СИСТЕМА Линия запуск: станков 5, контролёров 1, роботов 2, постов 1 - 10 процессов

Во время работы каждое событие выводится отдельной строкой: модельное время, вид события, процесс-участник и описание. Виды событий: СОЗДАНИЕ, ОЧЕРЕДЬ, ПЕРЕВОЗКА, ОП.НАЧАЛО, ОП.КОНЕЦ, КОНТРОЛЬ, БРАК, ПЕРЕДЕЛКА, СПИСАНИЕ, КОМПЛЕКТ, СБОРКА, ВЫПУСК, ПРОСТОЙ (станок или контролёр ждёт места) и СИСТЕМА (подключение и завершение процессов, запуск линии, получен

сигнал). Каждая строка выводится одним системным вызовом write, а журнал открыт с флагом O_APPEND, поэтому строки разных процессов не перемешиваются. Так как события печатаются внутри операций, их порядок совпадает с порядком изменений линии.

По сигналу SIGUSR1 печатается снимок состояния линии:

--- Состояние линии, t=25, процессов 10 ---
Выпуск: REDUCER 0/4
Склад: 6, вход ОТК: 0/3, выход ОТК: 0/2
Станки: LATHE-1[-] MILL-1[-] MILL-2[-] DRILL-1[-] GRIND-1[GEAR#5]
Роботы: РОБОТ-1[GEAR#2] РОБОТ-2[HOUSING#3]
Комплекты: SHAFT 2/4 GEAR 0/4 HOUSING 0/4

В конце при любом способе завершения выводятся итоги: причина завершения; время работы в модельных единицах и секундах и число процессов; число событий и использованное начальное значение генератора; число проверок и нарушений правил; выпуск по каждому изделию, средний цикл изготовления изделия и такт выпуска; по каждому типу детали - сколько создано, забраковано, переделано, списано, вошло в изделия и средний путь от склада до комплекта; по каждому процессу-участнику - число операций и доля времени работы и простоя; наибольшее заполнение склада и накопителей ОТК. Параметры, события и итоги одновременно записываются в журнал.

Варианты завершения программы

Каждому варианту соответствует свой код завершения. Во всех вариантах, где объекты IPC уже созданы, они удаляются перед выходом.

План выполнен - все изделия по плану выпущены. Последний сборочный пост останавливает линию. Код 0.

Ошибка в данных - неверный ключ, файл не найден или в нём ошибка. Процессы и объекты IPC не создаются. Код 1.

Исходные детали исчерпаны. Линия затихла, а для какого-то типа детали заготовок не осталось, и до плана не хватает. Это определяет сторож. Код 2.

Дальнейшее производство невозможно. Невозможность может быть видна по исходным данным (нет нужного станка, комплект не помещается в накопитель и т. п.) - тогда процессы не создаются. Либо линия может встать в процессе работы: детали заполнили накопители так, что все участники ждут друг друга (взаимная блокировка модели). Сторож замечает, что событий давно нет, а детали остались на линии, и перечисляет заблокированных участников. Процессы при этом не зависают: они ждут на семафорах и завершаются по команде менеджера. Третья ситуация возможна у независимых процессов: если участник завершился досрочно (ему послали SIGTERM), линия без него встаёт, и менеджер сообщает, сколько участников осталось. Код 3.

Превышен лимит модельного времени (ключ -t). Код 4.
Нарушены правила модели - возможно только при ошибке в программе. Код 5.

Прерывание сигналом SIGINT (Ctrl+C) или SIGTERM. Линия останавливается, все процессы-участники завершаются, итоги выводятся и записываются в журнал. Коды 130 и 143.

Мягкая остановка по сигналу SIGUSR2. Склад перестаёт выпускать заготовки, детали в работе доделываются, после чего программа завершается. Код 140. Если оставшихся на линии деталей хватило на выполнение плана, итог - «план выполнен», код 0.

Если менеджер убит сигналом SIGKILL, который перехватить нельзя, объекты IPC остаются в системе. При следующем запуске с тем же именем менеджер удаляет их перед созданием новых.

Структура программы

Файлы config.h и config.c - параметры, встроенная конфигурация, чтение и проверка файла. Файл model.h - структуры линии и участников, номера операций и семафоров. Файл line.c - операции участников над линией, сторож, проверка правил, итоги. Файл worker.c - поведение процессов-участников, общее для всех программ. Файлы sync_unnamed.c и sync_named.c - две реализации семафоров с одинаковым интерфейсом sync.h. Файлы call_shm.c и call_mq.c - два способа выполнить операцию: под замком в разделяемой памяти или через очереди сообщений. Файл main_shm.c - родитель и менеджер для программ 1-3, manager_mq.c - менеджер программы 4, worker_main.c - независимый участник для программ 3 и 4, app.c - разбор ключей, журнал и сигналы менеджера, log.c - вывод событий.

Результаты работы

Базовый сценарий

Прогон ./bin/line_unnamed -c configs/01_basic.cfg -d 20. Фрагмент вывода, в котором видны приостановка станка, брак, списание и переделка:

[t= 50] ПРОСТОЙ GRIND-1 выходной накопитель заполнен - станок приостановлен, GEAR#8 на станке
[t= 53] ПРОСТОЙ GRIND-1 место освободилось - станок возобновляет работу
[t= 53] БРАК ОТК-1 в HOUSING#9 обнаружен брак
[t= 53] СПИСАНИЕ ОТК-1 HOUSING#9 списана: переделка не предусмотрена
[t= 56] БРАК ОТК-1 в SHAFT#10 обнаружен брак
[t= 56] ПЕРЕДЕЛКА ОТК-1 SHAFT#10 возвращается на операцию 2 (GRIND), переделка 1 из 1
...
[t= 95] СБОРКА ПОСТ-1 начало сборки REDUCER №4 из комплекта: SHAFT#10, GEAR#15, GEAR#16, HOUSING#17
[t= 100] ВЫПУСК ПОСТ-1 выпущено изделие REDUCER №4, по плану 4/4
[t= 100] СИСТЕМА РОБОТ-1 процесс завершился
[t= 100] СИСТЕМА MILL-1 процесс завершился

...

Итоги:

Завершение: ПЛАН ВЫПОЛНЕН
Причина: все изделия по плану выпущены
Время работы: 102 ед. (2.0 с), процессов участников: 10
Событий: 491, seed: 13
Проверок правил: 138, нарушений правил модели: 0
Изделие REDUCER: выпущено 4 из 4
Средний цикл изделия: 63.2, такт выпуска: 25.5
Деталь SHAFT: создано 4, брак 1, переделок 1, списано 0, в изделиях 4, на линии 0, ср. путь до комплекта 28.5
Деталь GEAR: создано 8, брак 1, переделок 1, списано 0, в изделиях 8, на линии 0, ср. путь до комплекта 44.5
Деталь HOUSING: создано 5, брак 1, переделок 0, списано 1, в изделиях 4, на линии 0, ср. путь до комплекта 30.2
Станок LATHE-1: операций 4, работа 15.6%, приостановлен 0.0%
Станок MILL-1: операций 7, работа 35.2%, приостановлен 0.0%
Станок MILL-2: операций 6, работа 33.2%, приостановлен 0.0%
Станок DRILL-1: операций 14, работа 32.3%, приостановлен 1.0% Станок GRIND-1: операций 14, работа 32.3%, приостановлен 9.8% Контролёр ОТК-1: проверено 19, брак 3, работа 18.7%, ожидание места 0.0% Робот РОБОТ-1: рейсов 40, работа 78.3%
Робот РОБОТ-2: рейсов 40, работа 78.2%
Пост ПОСТ-1: собрано 4, работа 19.6%
Максимальное заполнение: склад 8, вход ОТК 1/3, выход ОТК 1/2

Вал SHAFT#10 был забракован, вернулся на шлифовку и вошёл в четвёртый редуктор. Списанный корпус HOUSING#9 заменён новой заготовкой (создано 5 корпусов при плане 4). Узкое место линии, как и в предыдущих заданиях, - транспорт: роботы заняты около 78 % времени, станки - 16-35 %. Шлифовальный станок GRIND-1 почти 10 % времени простоял с готовой деталью, потому что роботы не успевали её забрать.

Сравнение четырёх программ

Тот же сценарий с той же задержкой (-d 20) во всех четырёх программах выполнил план, нарушений правил не было:

line_unnamed: время 102 ед., событий 491, брак 3, роботы заняты 78 %
line_named: время 107 ед., событий 485, брак 3, роботы заняты 73-77 % shm (независимые процессы): время 102 ед., событий 479, брак 3, роботы заняты 76-80 %
mq (очереди сообщений): время 94 ед., событий 463, брак 2, роботы заняты 81-83 %

Числа немного различаются: процессы планируются операционной системой,

поэтому порядок, в котором детали попадают к контролёру, а значит, и то, какой детали достанется брак, каждый раз может быть другим. Это ожидаемое свойство модели из параллельных процессов, итог - выполнение плана - при этом стабилен.

Независимые процессы на очередях сообщений

Прогон ./tests/run_independent.sh mq -c configs/01_basic.cfg -d 20. Менеджер сообщает, кого ждёт, участники подключаются в произвольном порядке, и после подключения последнего линия запускается:

Менеджер asm_line запущен. Ожидаются процессы-участники (10):
./bin/mq_worker -n asm_line <роль> <номер>
[t= 0] СИСТЕМА Склад процесс подключился (1 из 10)
[t= 0] СИСТЕМА LATHE-1 процесс подключился (2 из 10)
...
[t= 0] СИСТЕМА ПОСТ-1 процесс подключился (10 из 10)
[t= 0] СИСТЕМА Линия запуск: станков 5, контролёров 1, роботов 2, постов 1 - 10 процессов
...
[t= 48] БРАК ОТК-1 в SHAFT#10 обнаружен брак
[t= 48] ПЕРЕДЕЛКА ОТК-1 SHAFT#10 возвращается на операцию 2 (GRIND), переделка 1 из 1
[t= 53] БРАК ОТК-1 в HOUSING#9 обнаружен брак
[t= 53] СПИСАНИЕ ОТК-1 HOUSING#9 списана: переделка не предусмотрена
...
[t= 92] СИСТЕМА MILL-2 процесс завершился
[t= 92] СИСТЕМА ПОСТ-1 процесс завершился
[t= 94] СИСТЕМА Склад процесс завершился

Во время работы в системе существуют 35 именованных семафоров, очередь запросов и 10 очередей ответов; разделяемой памяти нет. После завершения все они удалены.

Сигналы

Мягкая остановка в программе line_unnamed: сначала послан SIGUSR1 (выведено состояние, приведённое в разделе «Что выводит программа»), затем SIGUSR2.

[t= 35] СИСТЕМА Линия SIGUSR2: новые заготовки не выпускаются, доделываем детали в работе
...
Завершение: МЯГКАЯ ОСТАНОВКА (SIGUSR2)
Причина: детали, которые были в работе, доделаны
Изделие REDUCER: выпущено 3 из 4

Код завершения 140. Из деталей, которые уже были на линии, собрано три редуктора.

Прерывание Ctrl+C в программе line_named (SIGINT послан всей группе процессов, как это делает терминал):

[t= 25] СИСТЕМА Линия получен сигнал SIGINT - остановка
[t= 25] СИСТЕМА ОТК-1 процесс завершился
[t= 25] СИСТЕМА ПОСТ-1 процесс завершился
...
Завершение: ПРЕРВАНО СИГНАЛОМ
Причина: получен сигнал SIGINT (Ctrl+C)
Время работы: 27 ед. (0.8 с), процессов участников: 10
Проверок правил: 55, нарушений правил модели: 0

Код завершения 130. Все процессы-участники завершились, итоги записаны в журнал, разделяемая память и семафоры удалены.

Досрочное завершение участника в программе mq: процессу DRILL-1 послан SIGTERM.

[t= 26] СИСТЕМА DRILL-1 процесс завершился
...
Завершение: ПРОИЗВОДСТВО НЕВОЗМОЖНО
Причина: процессов-участников осталось 9 из 10

Код завершения 3.

Исчерпание заготовок

Сценарий configs/04_raw_exhausted.cfg - заготовок PLATE меньше, чем нужно по плану:

Завершение: ИСХОДНЫЕ ДЕТАЛИ ИСЧЕРПАНЫ
Причина: заготовки PLATE исчерпаны (списано 2)

Взаимная блокировка

Сценарий configs/05_deadlock.cfg. Быстрые детали LEFT заполняют комплект, выход и вход ОТК, и медленная деталь RIGHT, без которой комплект не собрать, уже не может попасть на контроль. Сторож замечает, что линия встала:

Завершение: ПРОИЗВОДСТВО НЕВОЗМОЖНО (взаимная блокировка)
Причина: на линии застряло деталей: 9; WELD-1 приостановлен; ОТК-1 ждёт места.

Тестирование

Используются те же десять тестовых наборов данных, что и в ИДЗ-1 и ИДЗ-2 (папка configs): работа с браком, переделкой и списанием; работа без брака; высокий брак с многократной переделкой; исчерпание заготовок; взаимная блокировка; отсутствие нужного станка; комплект, не помещающийся в накопитель; ошибка в

файле; узкое место с приостановкой станков; два вида изделий с общей деталью и двумя постами.

Скрипт tests/run_tests.sh запускается командой make test и проверяет все четыре программы. Для каждого прогона проверяются код завершения, итоговое сообщение, отсутствие нарушений правил и то, что после завершения в системе не осталось разделяемой памяти, семафоров и очередей сообщений. Кроме того, скрипт проверяет:

- встроенную конфигурацию, лимит времени, несуществующий файл и неверный ключ;
- что в журнале базового сценария есть все обязательные виды событий и что все 10 процессов подключились и завершились;
- что при высокой вероятности брака есть переделки;
- реакцию на сигналы: SIGINT всей группе процессов (код 130), SIGTERM менеджеру (код 143), SIGUSR1 (выводится состояние), SIGUSR2 (мягкая остановка, код 140);
- для независимых процессов - досрочное завершение участника (код 3).

Все 86 проверок проходят. Дополнительно проверено вручную: независимые участники запускаются в любом порядке, в том числе раньше менеджера; второй участник с тем же номером и участник с несуществующим номером не подключаются; программы многократно запускались с очень коротким тактом (-d 2), когда процессы особенно активно конкурируют за семафоры, - зависаний и нарушений правил не было.